



Apresentação

Segundo Ramakrishnan (2011, p. 435), uma transação é definida como qualquer execução única de um programa de usuário em um SGBD e difere da execução de um programa fora do SGBD em aspectos importantes. Isso quer dizer que **uma transação é um bloco único e indivisível (atômico), e é independente de outras transações e processos do SGBD.**

Uma transação pode ser qualquer alteração feita no SGBD, ou seja, a execução de um comando de SELECT, INSERT, UPDATE ou DELETE é considerada transação. Assim, as transações tratam de aspectos importantes do SGBD, como concorrência e recuperação de falhas.

Nesta Unidade de Aprendizagem, você vai aprender mais sobre transações em SGBDs, por que elas são importantes e como elas impactam no desenvolvimento dos seus sistemas.

Bons estudos.

Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Explicar o que são transações em banco de dados.
- Relacionar as propriedades das transações em banco de dados.
- Aplicar transações em SQL.



Desafio

Uma transação simboliza uma unidade de trabalho executada dentro do SGBD e tratada de maneira coerente e confiável, independentemente de outras transações. Com isso, as transações exercem um importante papel, podendo ser utilizadas em diversos cenários para muitas ocasiões, desde sistemas grandes, como em sistemas bancários, até pequenos sistemas de ponto de venda, sendo que o principal motivo da utilização de transações é para ter mais controle sobre o que acontece no banco de dados.

Acompanhe a situação:

Imagine que você foi contratado para fazer a manutenção de um sistema de controle de estoque de um mercado. No mercado são realizadas vendas de produtos alimentícios e, com isso, é dado baixa no estoque dos produtos.

As vendas de produtos desse mercado são registradas no SGBD para controle interno, gerar relatórios, etc. Além disso, são guardadas informações com relação à data de vencimento do produto na Tabela Produto. Veja:

TABELA - PRODUTO				
ID	NOME	VALOR		
1	Leite	3.75	12/02/2020	ATIVO
2	Refrigerante	5.50	05/01/2020	ATIVO
3	Ovos (dúzia)	6.00	23/11/2020	ATIVO

A tabela mostra que os produtos vencidos não estão tendo seu STATUS atualizado para INATIVO.

Considerando esse cenário, crie uma transação explícita no banco de dados Postgre que selecione todos os produtos vencidos e troque o status deles para inativo.



No geral, os **SGBDs transacionais**, como o **PostgreSQL**, trabalham com o conceito de **ACID** (atomicidade, consistência, isolamento e durabilidade), além de **trabalharem por padrão com o segundo nível de isolamento (READ COMMITED)**. Sendo assim, é possível controlar de forma rigorosa o paralelismo e a concorrência no banco de dados.

No entanto, um controle muito rigoroso de concorrência gera outros problemas no SGBD e, por isso, é importante entender quando utilizar esse recurso.

Neste Infográfico, você vai ver dois problemas que podem ser causados pelo alto controle de concorrência em transações.

PROBLEMAS DO ALTO CONTROLE DE CONCORRÊNCIA EM TRANSAÇÕES

A má utilização no controle de concorrência de transações em SGBDs tende a causar problemas graves, que podem ser difíceis de resolver, e, por isso, é muito importante entender exatamente quando isso deve ser feito.

VEJA AGORA OS PROBLEMAS QUE PODEM ACONTECER:

> Lentidão

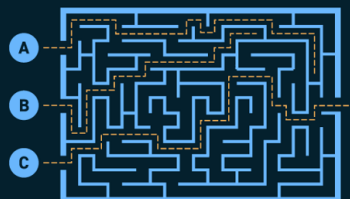
Suponha que existam as Transações 1 e 2 acontecendo no SGBD no modo *serializable*. Nesse caso, as transações fazem um bloqueio de compartilhamento das informações que a transação necessita, ou seja, outras transações que precisem do mesmo dado terão que esperar ela terminar.

Dessa forma, quando a Transação 1 for executada, ela vai bloquear as informações que ela precisa. E a Transação 2, que precisa mexer nas mesmas informações, vai ter que esperar a Transação 1 terminar para depois ser executada. Logo, se muitas transações necessitarem da mesma informação, o SGBD vai formar uma "fila" de espera para execuções, o que acarretará em lentidão nas respostas do banco de dados e poderá atrapalhar a aplicação que consome esses dados.



CARREGANDO...

> Deadlock (Impasse)



Um *deadlock* acontece quando a Transação 1 precisa dos dados da Transação 2 para terminar, assim como a Transação 2 precisa dos dados da Transação 1 para terminar, gerando um ciclo em que nenhuma das transações pode ser terminada, resultando em gargalos no SGBD. Quando o nível de isolamento da transação está em *serializable*, a transação bloqueia o compartilhamento de recursos, gerando muito mais *deadlocks*.

Agora que você conhece alguns problemas na utilização de alto controle de transações em SGBDs, poderá tomar decisões mais conscientes sobre o que pode acontecer no futuro com o seu banco de dados e sua aplicação. A princípio, o uso do nível de isolamento mais alto pode parecer uma boa solução, por prevenir alguns problemas, no entanto, ele pode causar outros. Por isso, é importante tomar muito cuidado ao mudar o nível de isolamento de uma transação.





Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Conteúdo do livro

As **transações em SGBDs** são processos importantes de se conhecer, pois dizem **respeito a todas as operações realizadas no banco de dados**, como: insert, update, delete e consultas (select).

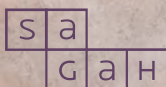
As aplicações que utilizam o SGBD também fazem transações e, com isso, conhecer o funcionamento das transações faz com que as aplicações tenham mais liberdade de uso dentro do SGBD e consigam se adaptar a diferentes cenários e trabalhar melhor com concorrência de processos.

No capítulo **Transações em banco de dados**, da obra *Consulta em Banco de Dados*, você vai conhecer mais sobre as transações em banco de dados e seu comportamento.

Boa leitura.

CONSULTAS EM BANCO DE DADOS

Fellipe Soares de Oliveira



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Transações em banco de dados

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Explicar o que são transações em banco de dados.
- Relacionar as propriedades das transações em banco de dados.
- Aplicar transações em Structured Query Language (SQL).

Introdução

Uma transação em banco de dados é um programa em execução ou processo que inclui um ou mais acessos ao banco de dados, como a leitura ou a atualização de registros (ELMASRI; NAVATHE, 2005), por exemplo, as execuções de comandos no Sistema de Gerenciamento de Banco de Dados (SGBD) como um `INSERT` ou `UPDATE`, além de *triggers*, *procedures* e funções.

De acordo com Ramakrishnan e Gehrke (2008), as transações em SGBD auxiliam no gerenciamento de execuções paralelas, na recuperação de falhas de transações não concluídas e na intercalação de execuções de comandos, abrangendo, assim, aspectos muito importantes com os quais o usuário geralmente não se preocupa, pois o SGBD gerencia tudo por “debaixo dos panos”.

Portanto, entender o que são as transações, como elas se comportam e quais são os seus efeitos é essencial para construir uma aplicação com um bom desempenho e que faça bom uso dos recursos oferecidos pelo SGBD.

Neste capítulo, você conhecerá mais sobre transações em bancos de dados, suas propriedades e como executá-las.

1 Definição

Quando tratamos de concorrência, segundo Elmasri e Navathe (2005), os sistemas SGBD podem considerados monousuários, nos quais somente um usuário pode usar o sistema de cada vez, e multiusuários, quando muitos usuários podem utilizar o SGBD de forma concorrente. Com isso, o SGBD deve dispor de mecanismos de prevenção de falhas e gerenciamento de processos. Por exemplo, tenha em conta o seguinte cenário:

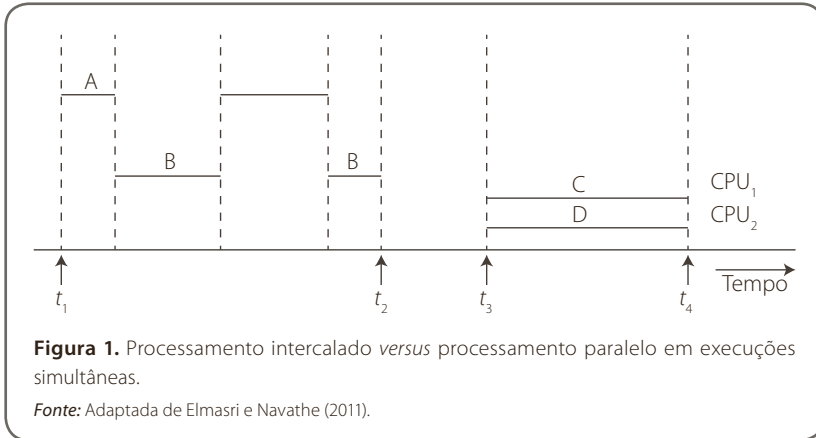
1. O usuário João executa um comando `UPDATE` na tabela funcionários do registro número 15 do banco de dados.
2. Ao mesmo tempo que João está fazendo o `UPDATE`, Maria está consultando a tabela funcionários, cujo resultado devolverá o registro número 15 que João estava alterando.

Como o SGBD deve se comportar se chegarem ao mesmo tempo duas transações (uma de consulta e outra de atualização) envolvendo a mesma tabela e o mesmo registro no SGBD?

O gerenciamento de transações de SGBD tem por objetivo resolver esses conflitos que, às vezes, passam despercebidos pelos usuários, mas são muito comuns no dia a dia.

Segundo Elmasri e Navathe (2011), os SGBD são construídos para lidar com duas formas diferentes de resolver esse problema de concorrência: a intercalação de processos, em que o SGBD se alterna na execução dos processos, ou seja, executa alguns comandos do primeiro processo e, depois, muda para a execução de alguns comandos do segundo processo, e assim por diante até terminar a execução; e o paralelismo, somente possível quando o computador em que o SGBD está tem mais de um núcleo de processador (algo bem comum nos computadores atuais), caso em que os processos são executados de maneira paralela, mas em núcleos diferentes.

A Figura 1 mostra o cenário dos processamentos intercalado (processos A e B) e paralelo (processos C e D), tornando visíveis a alternância das execuções dos processos A e B, e a simultaneidade dos processos C e D.



Elmasri e Navathe (2011, p. 505) afirmam que:

[...] sempre que uma transação é submetida em um SGBD para execução, o sistema fica responsável por garantir que todas as operações na transação sejam concluídas com sucesso e seu efeito seja registrado permanentemente no banco de dados, ou que a transação não tenha qualquer efeito no banco de dados ou quaisquer outras transações.

Assim, uma transação executada corretamente no SGBD é considerada confirmada (*committed*, ou seja, com `commit`) ou é abortada. Ainda segundo Elmasri e Navathe (2011), um SGBD precisa garantir a execução integral de uma transação com sucesso, sem que partes da transação sejam executadas e outras não. Quando da execução de uma transação com cinco operações, em que as duas primeiras são feitas com sucesso e a terceira falhar, o SGBD deve garantir que as operações feitas sejam revertidas e que não provoquem nenhum efeito no banco de dados.

Níveis de isolamento em transações

Segundo Elmasri e Navathe (2005), o isolamento é imposto pelo subsistema de controle de concorrência do SGBD. Se alguma transação não tornar suas atualizações invisíveis às outras até sua efetivação, será imposta alguma forma de isolamento que solucione o problema de atualizações temporárias e elimine reversões (comando `rollback`) em cascata. Isso quer dizer que o nível de

isolamento do SGBD que geralmente pode ser configurado pelo usuário está atrelado à quantidade de concorrência que o SGBD permite e gerencia.

De acordo com Ramakrishnan e Gehrke (2008), existem quatro níveis de isolamento, em que o nível mais baixo permite total concorrência, além do fato de que, quanto maior o nível, o SGBD possibilita menos concorrência. Os níveis de isolamento servem para evitar problemas de concorrência, quando o SGBD transfere a responsabilidade de controle para o usuário. Os níveis de isolamento são:

1. `READ UNCOMMITTED` (leitura de dados não efetivada): nível menos isolado e que possibilita a leitura de uma transação antes que esta seja confirmada, ou seja, permite que uma transação A ainda não efetivada (*committed*) seja lida (consultada) por uma transação B e, depois dessa leitura, a transação A pode sofrer um `rollback` e a alteração ser desfeita (i. e., a transação B leu um dado da transação A que não existe mais, porque foi desfeito). Esse tipo de problema é chamado *dirty reads* (leituras sujas).
2. `READ COMMITTED` (leitura efetivada): o segundo nível de isolamento é o nível-padrão da maioria dos SGBD, inclusive o PostgreSQL. Não promove problemas do tipo *dirty reads*, porém permite os *non-repeatable reads* (leituras não repetíveis), que acontecem quando uma transação A altera o valor de um objeto do banco de dados B que foi lido por uma transação C, alteração que se deu enquanto a transação C lia o objeto B. Assim, se a transação C ler novamente o objeto B, ela terá um resultado diferente do primeiro, porque este já foi alterado.
3. `REPEATABLE READ` (leitura repetível): embora o terceiro nível não permita que aconteçam os problemas de *dirty reads* e *non-repeatable reads* dos níveis 1 e 2, podem acontecer os chamados *phantoms* (fantasmas). Estes surgem quando uma transação A poderia sobrescrever o valor de um objeto B, já modificado por uma transação C, enquanto a transação C ainda está em andamento, mesmo que a transação A não leia o valor do objeto B gravado pela transação C. Isso quer dizer que a transação A sobrescreve o dado alterado na transação C antes de a transação C terminar, ou seja, a alteração da transação C não surtirá efeito porque foi sobrescrita.

4. **SERIALIZABLE** (serializável): único que garante que nenhum dos três problemas citados ocorrerá. Isso porque as transações são executadas de modo sequencial, e não mais de modo concorrente, o que pode deixar as operações mais lentas. O único modo de executar operações de modo concorrente nesse nível é se as transações concorrentes não usarem nenhum dado compartilhado entre elas.

Os tipos de isolamento retratados são comuns nos SGBD e os casos apresentados em cada um deles nem sempre representam de fato um problema a ser solucionado de imediato — por isso, o SGBD permite que o usuário modifique o tipo de isolamento da transação, para que ele tenha mais controle sobre o SGBD e consiga desenvolver aplicações de diferentes tipos com diferentes necessidades, às quais o banco de dados se adapta.

Para mudar o nível de isolamento de uma transação, basta adicionar o comando a seguir:

```
BEGIN
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
    INSERT INTO funcionario(id, nome, idade, endereco, salario,
status _ salario)
    VALUES(1, 'João Silva', 23, 'Avenida Paulista, 1000', 1.500,
'PAGO');
    COMMIT;
```

A linha 2 deve seguir após o comando **BEGIN**, e, no final, indica-se qual o nível de isolamento você quer utilizar: **SERIALIZABLE**, **REPEATABLE READ**, **READ COMMITTE** ou **READ UNCOMMITTED**.



Fique atento

Segundo Elmasri e Navathe (2011), as consultas (comando **SELECT**) realizadas no SGBD são conhecidas como transações somente de leitura. Os demais comandos, como **INSERT**, **UPDATE** e **DELETE**, compreendem transações do tipo leitura-gravação.

2 Propriedades das transações em banco de dados

Para Ramakrishnan e Gehrke (2008), uma transação constitui uma execução de um programa de usuário, vista pelo SGBD como uma série de operações de leitura e gravação. Assim, um SGBD deve garantir quatro propriedades importantes das transações para manter os dados consistentes mediante o acesso concorrente e falhas de sistema. Essas propriedades levam o acrônimo de ACID, que significa “atomicidade, consistência, isolamento e durabilidade”:

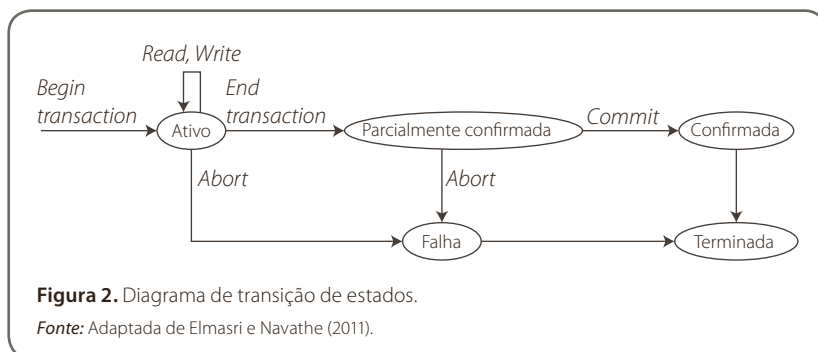
1. **Atomicidade:** os usuários devem conseguir enxergar a execução de cada transação como atômica — todas as ações são executadas ou nenhuma delas é executada. Os usuários não devem se preocupar com o efeito de transações incompletas, como quando ocorre uma falha de sistema (RAMAKRISHNAN; GEHRKE, 2008). Portanto, a atomicidade está ligada ao fato de o SGBD ser capaz de executar uma transação completamente com sucesso ou não executar nada, garantindo, assim, a atomicidade.
2. **Consistência:** cada transação, executada isoladamente, sem nenhuma execução concorrente de outras transações, deve preservar a consistência do banco de dados. O SGBD presume que a consistência é preservada por transação. Garantir essa propriedade de uma transação é responsabilidade do usuário (RAMAKRISHNAN; GEHRKE, 2008). O SGBD precisa criar um estado válido dos dados em caso de sucesso na transação; e em caso de falhas, a transação deve retornar ao seu estado antes da alteração, garantindo, assim, a consistência.
3. **Isolamento:** os usuários devem ser capazes de entender uma transação sem considerar o efeito de outras transações em execução concorrente, mesmo que o SGBD intercale as ações de várias transações por motivos de desempenho. Algumas vezes, essa propriedade é chamada de isolamento: as transações são isoladas (ou protegidas) dos efeitos do plano de execução concorrente de outras transações (RAMAKRISHNAN; GEHRKE, 2008). Assim, uma transação em andamento, mas ainda não validada, deve permanecer isolada de qualquer outra operação, garantindo-se que a transação não sofrerá interferência de nenhuma outra transação concorrente, assegurando-se, assim, o isolamento das transações.

4. **Durabilidade:** uma vez que o SGBD informe ao usuário que uma transação foi concluída com êxito, seus efeitos devem persistir, mesmo que o sistema falhe antes que todas as suas alterações sejam refletidas no disco. Essa propriedade é chamada de durabilidade (RAMAKRISHNAN; GEHRKE, 2008), ou seja, os dados validados são registrados pelo sistema de tal modo que, mesmo no caso de uma falha e/ou do reinício do sistema, os dados estão disponíveis em seu estado correto.

Portanto, as características das propriedades ACID possibilitam que os desenvolvedores e as aplicações que utilizam o SGBD não se preocupem tanto com a complexidade do SGBD, pois todo o gerenciamento de dados é realizado por ele; assim, os desenvolvedores podem se concentrar em desenvolver a lógica da aplicação em vez de se ater à integridade dos dados do SGBD ou ao fato de algum erro estar acontecendo em cada transação, o que resulta em mais liberdade e confiabilidade.

Estados de transações em banco de dados

Conforme Elmasri e Navathe (2011), as transações compreendem unidades atômicas de trabalho e necessariamente precisam ser concluídas ou abortadas e não afetar o banco de dados de nenhum modo. Por isso, para fins de recuperação de falhas, o SGBD deve registrar os estados de cada transação: início da transação, final da transação, transação confirmada ou transação abortada. Na Figura 2, você pode observar um diagrama de transição de estados que demonstra o fluxo de informações de uma transação no SGBD.



A partir do conhecimento das partes que compõem uma transação, veja agora o que significa cada parte do diagrama:

1. **BEGIN_TRANSACTION**: marca o início da execução de uma transação no banco de dados.
2. **READ** ou **WRITE**: especificam operações de leitura ou gravação nos itens do banco de dados executados como partes de uma transação.
3. **END_TRANSACTION**: especifica que operações de transações como **READ** e **WRITE** terminaram e marcam o final da execução da transação. Entretanto, nesse ponto pode se tornar necessário verificar se as mudanças feitas pela transação podem ser permanentemente aplicadas ao banco de dados ou se a transação precisa ser abortada por algum motivo (p. ex., erro em alguma operação da transação).
4. **COMMIT_TRANSACTION**: sinaliza um final bem-sucedido da transação, de modo que quaisquer mudanças no banco de dados podem ser seguramente confirmadas e não serão desfeitas.
5. **ROLLBACK** ou **ABORT**: sinalizam que uma operação foi encerrada sem sucesso, de modo que qualquer alteração realizada pela transação no banco de dados precisa necessariamente ser desfeita.

Segundo Elmasri e Navathe (2011), enquanto uma transação está sendo executada, o sistema guarda informações sobre ela nas tabelas do banco de dados — ao ser finalizada, essas informações são automaticamente removidas do sistema. Além disso, as transações com falhas/abortadas podem ser reiniciadas, automaticamente ou como uma nova transação executada pelo próprio usuário.



Saiba mais

É importante lembrar que a consistência dos dados também depende do usuário do banco de dados, ou seja, o usuário precisa garantir que o SGBD dispõe de uma rede, de um disco rígido para escrita dos dados, etc. Além disso, o usuário do banco de dados precisa garantir que o ambiente em que o SGBD está localizado é funcional tanto do ponto de vista físico quanto lógico, ou seja, o desenvolvedor deve criar lógicas que façam sentido no SGBD e não atrapalhem a consistência dos dados (RAMAKRISHNAN; GEHRKE, 2008).

3 Implementação de transações em SQL

Segundo a documentação oficial do PostgreSQL, as transações podem ser: explícitas, quando o usuário consegue controlar o início e o fim da execução da transação, além de poder fazer o `rollback` manualmente caso seja necessário, e implícitas, em que o usuário apenas executa o comando sem especificar o início e o fim da transação. Veja a seguir exemplos de transações explícita e implícita.

Transação explícita:

1. `BEGIN`: indica o início de uma transação de Structured Query Language (SQL).
2. Após o comando `BEGIN`, deve-se colocar os comandos que o usuário deseja que sejam executados, como `SELECT`, `INSERT`, `UPDATE` ou `DELETE`.
3. `COMMIT`: indica a confirmação de uma transação.
4. `ROLLBACK`: aponta o cancelamento de uma transação.
5. `END`: indica o encerramento do bloco de transação.

Nesse caso, você deve iniciar com o comando `BEGIN`, seguido da operação que deseja realizar no banco de dados, podendo ser inseridas diversas operações, e não somente uma. Depois, é necessário utilizar o comando `COMMIT/END` para finalizar e confirmar a transação ou o comando `ROLLBACK` para desfazer a alteração.

Transação implícita:

```
UPDATE funcionario SET salario = 2000 WHERE id_funcionario = 1;
```

Nesse tipo de transação, os comandos que iniciam e encerram a transação são omitidos, e a transação é confirmada automaticamente após sua execução. Assim, o usuário somente precisa executar a operação que ele quer realizar no banco de dados, sem se preocupar com seu início e fim ou um possível `ROLLBACK`.

Quando o usuário apenas executa um comando como o `UPDATE`, o `BEGIN` e o `COMMIT` estão implícitos e o usuário não controla o início e o fim da transação.

É importante lembrar que, quando você executa transações explícitas, deve executar o comando `COMMIT` para confirmar a transação; caso contrário, a transação não será efetivada, conforme vemos no exemplo a seguir.

Você criará uma tabela chamada `funcionario`:

```
CREATE TABLE funcionario
(
    id INT PRIMARY KEY NOT NULL,
    NOME TEXT NOT NULL,
    IDADE INT NOT NULL,
    ENDERECO CHAR(50),
    SALARIO REAL,
    STATUS _ SALARIO TEXT NOT NULL
);
```

Agora, fará uma transação:

```
BEGIN
    INSERT INTO funcionario(id, nome, idade, endereco, salario,
status _ salario)
    VALUES(1, 'João Silva', 23, 'Avenida Paulista, 1000', 1.500,
'PAGO');
COMMIT;
```

Com o comando `BEGIN`, você inicia uma transação, depois faz um `INSERT` na tabela `funcionario` e, por último, executa o comando `COMMIT`, que confirma a transação `INSERT` na tabela `funcionario`. Após a execução do comando `COMMIT`, a transação não pode ser desfeita.

Veja agora um exemplo de transação utilizando o comando `ROLLBACK`:

```
BEGIN
    INSERT INTO funcionario(id, nome, idade, endereco, salario,
status _ salario)
    VALUES(1, 'João Silva', 23, 'Avenida Paulista, 1000', 1.500,
'PAGO');
ROLLBACK;
```

A utilização do comando `ROLLBACK` em uma transação serve principalmente para validar uma alteração feita no SGBD, ou seja, você executa um `INSERT` no banco de dados e percebe que as informações foram inseridas de maneira incorreta, levando à necessidade de desfazer o comando, caso em que, após o `INSERT`, você pode executar o comando `ROLLBACK`, quando a alteração será desfeita.



Referências

ELMASRI, R.; NAVATHE, S. B. *Sistemas de bancos de dados*. 4. ed. [S. l.]: Pearson, 2005.

ELMASRI, R.; NAVATHE, S. B. *Sistemas de bancos de dados*. 6. ed. [S. l.]: Pearson, 2011.

RAMAKRISHNAN, R.; GEHRKE, J. *Sistemas de gerenciamento de banco de dados*. 3. ed. Porto Alegre: AMGH, 2008.

Leituras recomendadas

BARROS, P. *O que é ACID?* 2016. Disponível em: <https://medium.com/opensanca/o-que-%C3%A9-acid-59b11a81e2c6>. Acesso em: 22 abr. 2020.

LANHELLAS, R. *Transações no PostgreSQL: níveis de isolamento*. 2015. Disponível em: <https://www.devmedia.com.br/transacoes-no-postgresql-niveis-de-isolamento/32464>. Acesso em: 22 abr. 2020.

POSTGRE SQL. *Documentação do PostgreSQL 8.2.0*. [2019]. Disponível em: <http://pgdoc-pgtrbr.sourceforge.net/pg82/sql-begin.html>. Acesso em 10 de abril de 2020.



Fique atento

Os *links* para *sites da web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Dica do professor

A **execução de transações explícitas** permite que os usuários controlem o início e o fim de uma transação, além do seu nível de isolamento, o que faz com que as **aplicações que utilizam o SGBD** tenham mais liberdade em diversos cenários de utilização, podendo ter um controle maior ou menor da concorrência dos processos.

Nesta Dica do Professor, você aprenderá mais sobre a execução de transações no PostgreSQL.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Exercícios

- 1) Considerando as transações explícitas, onde o usuário determina o início e o fim da transação, qual dos comandos inicia, encerra e executa uma operação corretamente dentro de uma transação?
- A) BEGIN
INSERT INTO venda(id_venda, id_produto, valor) VALUES(1, 5, 34.20);
COMMIT;
- B) INSERT INTO venda(id_venda, id_produto, valor) VALUES(1, 5, 34.20);
COMMIT;
- C) BEGIN
INSERT INTO venda(id_venda, id_produto, valor) VALUES(1, 5, 34.20);
- D) BEGIN
COMMIT;
- E) INSERT INTO venda(id_venda, id_produto, valor) VALUES(1, 5, 34.20);
- 2) Considerando as transações explícitas, onde o usuário determina o início e fim da transação, qual dos comandos inicia, executa uma operação e aborta a operação executada?
- A) BEGIN
INSERT INTO funcionario(id_funcionario, nome, idade, salario) VALUES(1, 'João Paulo', 43, 3450.20);
COMMIT;
- B) BEGIN
INSERT INTO funcionario(id_funcionario, nome, idade, salario) VALUES(1, 'João Paulo', 43, 3450.20);
ROLLBACK;
- C) BEGIN
INSERT INTO funcionario(id_funcionario, nome, idade, salario) VALUES(1, 'João Paulo', 43, 3450.20);
END;

- D) `INSERT INTO funcionario(id_funcionario, nome, idade, salario) VALUES(1, 'João Paulo', 43, 3450.20);`
- E) `INSERT INTO funcionario(id_funcionario, nome, idade, salario) VALUES(1, 'João Paulo', 43, 3450.20);`
`COMMIT;`

- 3) **Existem muitos recursos que os SGBDs têm para dar flexibilidade nas transações e melhorar o desempenho das aplicações que consomem os dados do banco.**

Com isso, por que os SGBDs trabalham com intercalação de processos e paralelismo?

- A) Para não ter concorrência nas transações do SGBD.
- B) Para trabalhar com concorrência nas transações do SGBD.
- C) Para ter mais usuários no banco de dados.
- D) Para não ter erros nas transações do SGBD.
- E) Para executar *procedures* no banco de dados.

- 4) **Os níveis de isolamento de transações em banco de dados servem, principalmente, para o usuário poder controlar a concorrência e resolver problemas relacionados a isso.**

Com isso, qual transação está corretamente declarada para ser feita no nível de isolamento `READ UNCOMMITTED`?

- A) `BEGIN`
`SET TRANSACTION ISOLATION LEVEL UNCOMMITTED`
`INSERT INTO estoque(id_estoque, id_produto, estoque_produto) VALUES(1, 5, 3);`
`COMMIT;`
- B) `BEGIN`
`SET TRANSACTION ISOLATION LEVEL COMMITTED`
`INSERT INTO estoque(id_produto, estoque_produto) VALUES(1, 5, 3);`
`COMMIT;`
- C) `BEGIN`
`SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED`
`INSERT INTO estoque(id_estoque, id_produto, estoque_produto) VALUES(1, 5, 3);`
`COMMIT;`

D) BEGIN
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
INSERT INTO estoque(id_estoque, id_produto, estoque_produto) VALUES(1, 5, 3);
COMMIT;

E) BEGIN
INSERT INTO estoque(id_estoque, id_produto, estoque_produto) VALUES(1, 5, 3);
COMMIT;

5) O fluxo de execução de uma transação passa por diversas etapas até ser efetivamente confirmada e a alteração ser feita permanentemente no banco de dados.

Considerando as etapas de uma transação, qual transação segue o fluxo correto até ser confirmada?

A) BEGIN_TRANSACTION --> WRITE --> END_TRANSACTION --> ROLLBACK_TRANSACTION

B) WRITE --> END_TRANSACTION --> COMMIT_TRANSACTION

C) BEGIN_TRANSACTION --> WRITE --> COMMIT_TRANSACTION

D) BEGIN_TRANSACTION --> WRITE --> END_TRANSACTION --> COMMIT_TRANSACTION

E) BEGIN_TRANSACTION --> WRITE --> END_TRANSACTION --> COMMIT_TRANSACTION -> ROLLBACK_TRANSACTION



Na prática

Atualmente, existem muitas empresas que controlam os seus **processos de concorrência pelo Sistema de Gerenciamento de Banco de Dados (SGBD)**. Dessa forma, o SGBD fica responsável por garantir que um objeto no banco de dados só seja afetado por uma transação por vez. Nesse caso, as transações nesse bloco não interagem entre si e, portanto, o sistema impede que aconteça inconsistência nos dados.

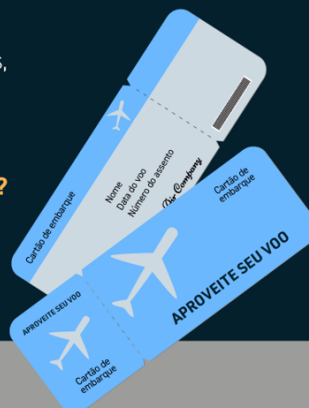
Veja, Na Prática, um estudo de caso de uma empresa de passagens aéreas que utiliza as transações do SGBD para controle de concorrência dos usuários na compra de passagens.

< O IMPACTO DAS TRANSAÇÕES EM SGBD DE UMA EMPRESA DE VENDA DE PASSAGENS ÁREAS >

Em empresas de venda de passagens aéreas, é possível que ocorra problemas de concorrência em compras de passagens, principalmente, em caso de promoções como a *Black Friday*. Isso quer dizer que mais de um usuário pode tentar comprar a mesma passagem ao mesmo tempo.

Como o sistema da empresa pode solucionar esse problema?

Uma das formas é fazendo um controle mais rigoroso de transações no SGBD. Acompanhe!



> Problema de venda de passagens aéreas

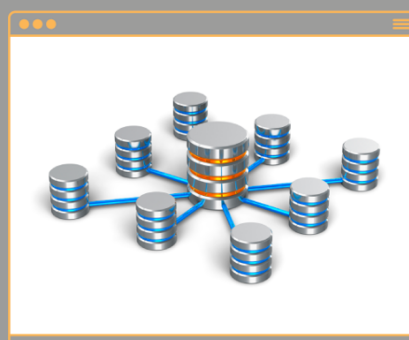
Um voo saiu de São Paulo com destino ao Rio de Janeiro em época de *Black Friday*. Esse voo teve uma promoção muito boa e, por conta disso, só tinha mais um lugar sobrando, que era a poltrona A20.



Nesse momento, três usuários diferentes tentaram comprar esse lugar sobrando. No entanto, o SGBD da companhia aérea estava habilitado como *serializable* e não permitia concorrência nesse mesmo bloco de dados. Isso significa que somente a transação do usuário que chegou primeiro (mesmo que por milésimos de segundos) foi executada, enquanto que nas outras, quando foram executadas, apareceu como assento já vendido.

> Qual foi a solução?

Para evitar problema de concorrência na compra de passagens, a empresa optou por **controlar rigorosamente a concorrência no SGBD**. Isso quer dizer que a empresa precisou habilitar um modo chamado *serializable*. Com isso, o SGBD não permitiu que mais de uma transação trabalhasse com o mesmo objeto no banco de dados.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Saiba +

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Transações em banco de dados

Confira, neste livro, especificamente no **Capítulo 16**, o assunto de gerenciamento de transações em bancos de dados e sua importância e cuidados.

Conteúdo interativo disponível na plataforma de ensino!

Transações e técnica para controle de concorrência

Este artigo fala sobre uma técnica chamada MVCC para o controle de concorrência no SGBD PostgreSQL.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Processamento de transações em SGBD

No vídeo a seguir, você verá mais detalhes sobre o processamento de transações em banco de dados.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Documentação oficial sobre transações do Postgre (em inglês)

No link a seguir, você vai conferir a documentação oficial do Postgre em inglês, podendo ser traduzida pelo próprio navegador. Este material é ideal para conhecer mais sobre transações nesse SGBD.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.